



INTERNET



ROUTE53

DEVOPS NETWORKING



COMPLETE QUICK REVISION GUIDE

- DNS
- TCP/IP
- HTTP
- HTTPS
- SSL/TLS
- CIDR 10.0.0.0/16
- VPC
- SUBNET
- ROUTING
- NAT
- VPN

INGRESS

LOAD BALANCER

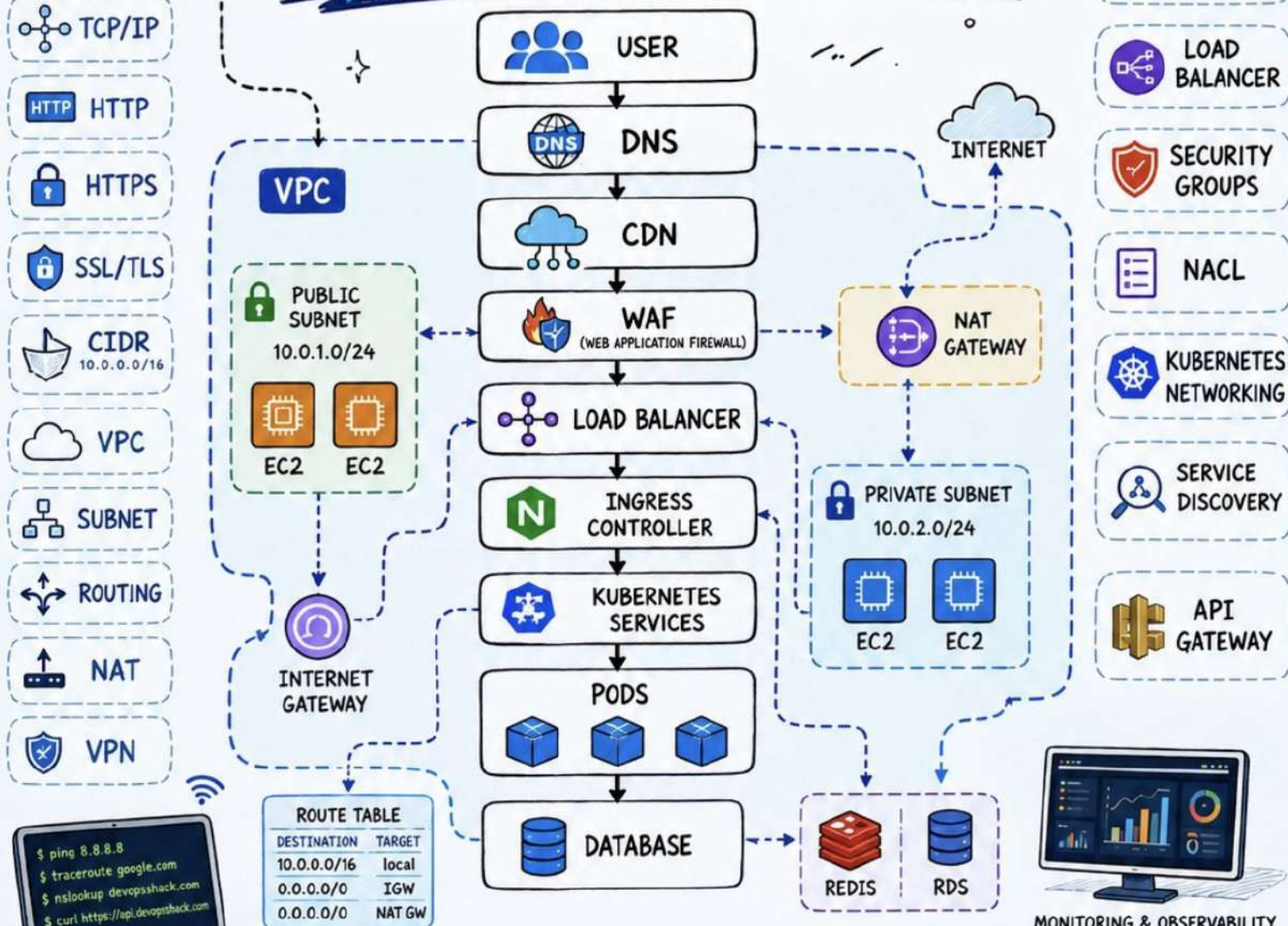
SECURITY GROUPS

NACL

KUBERNETES NETWORKING

SERVICE DISCOVERY

API GATEWAY



EVERYTHING A DEVOPS ENGINEER MUST KNOW ABOUT NETWORKING



DEVOPS SHACK

PRODUCTION-GRADE DEVOPS
CLOUD • KUBERNETES • DEVSECOPS

LEARN
BUILD
DEPLOY



02
/10

NETWORKING FUNDAMENTALS

Networking Basics Every DevOps Engineer Must Know

2/10

DEVOPS
SHACK

IP ADDRESS

Unique identity of a device on a network.

Example

192.168.1.10

IPv4 / IPv6



PUBLIC IP

Accessible from the internet.

Example

3.110.25.11

Routable on Internet



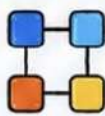
PRIVATE IP

Used for internal communication.

Example

10.0.0.15

Not routable on Internet



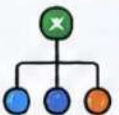
CIDR (CLASSLESS INTER-DOMAIN ROUTING)

Defines the size of the network and host space.

Example

10.0.0.0/16

65536 IPs (10.0.0.1 - 10.0.255.254)



SUBNET

Divides a network into smaller logical networks.

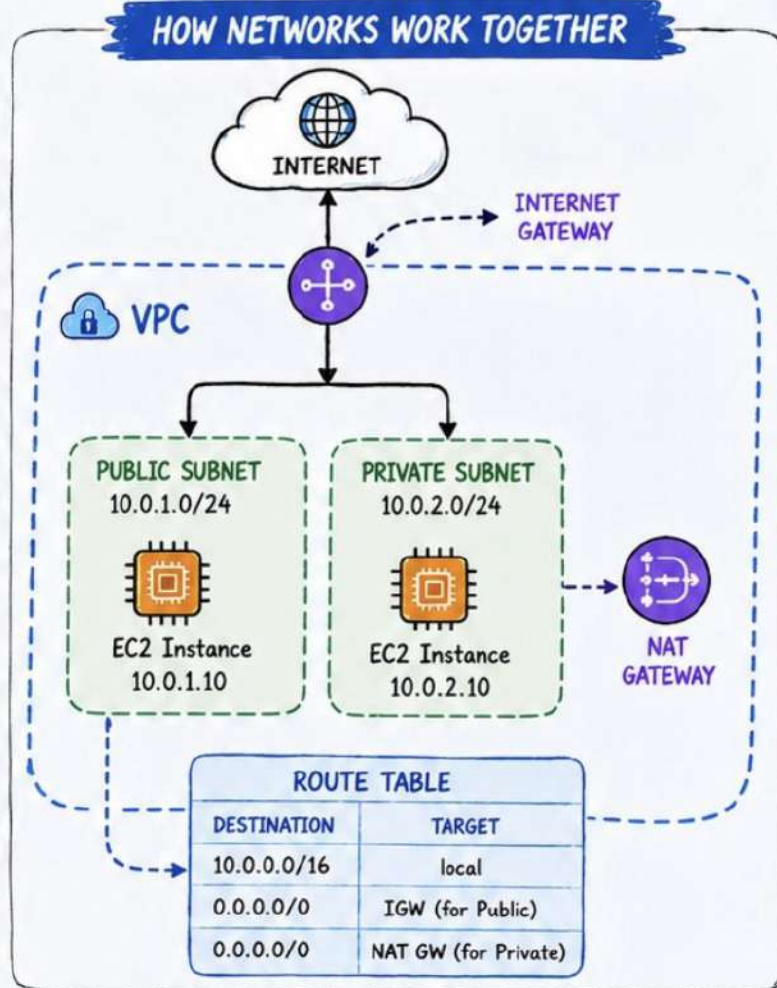
Examples

10.0.1.0/24

10.0.2.0/24

256 IPs each

HOW NETWORKS WORK TOGETHER



COMMON PROTOCOLS

- TCP** Transmission Control Protocol
Reliable, connection-oriented
- UDP** User Datagram Protocol
Fast, connectionless
- ICMP** Internet Control Message Protocol
Used for diagnostics (ping, traceroute)
- DNS** Domain Name System
Translates domain names to IPs
- HTTP** HyperText Transfer Protocol
Web traffic (port 80)
- HTTPS** HTTP Secure (SSL/TLS)
Secure web traffic (port 443)

KEY TAKEAWAYS

- ✓ Everything in a network communicates using IP.
- ✓ Private IPs are used inside networks, Public IPs are used on the internet.
- ✓ CIDR helps in efficient IP allocation.
- ✓ Subnets improve security, performance & organization.
- ✓ Route Tables decide where traffic should go.

QUICK TIP

Use private IPs for internal resources and expose only what's necessary to the internet.



Strong networking foundation makes you a better DevOps Engineer!



DEVOPS SHACK

PRODUCTION-GRADE DEVOPS • CLOUD • KUBERNETES • DEVSECOPS

STAY CONNECTED

- /devopsshack
- @devopsshack
- www.devopsshack.com

How Domain Name System Works

WHAT IS DNS?



DNS (Domain Name System) is like the internet's phonebook. It converts human-readable domain names into IP addresses that computers understand.

EXAMPLE

When you type

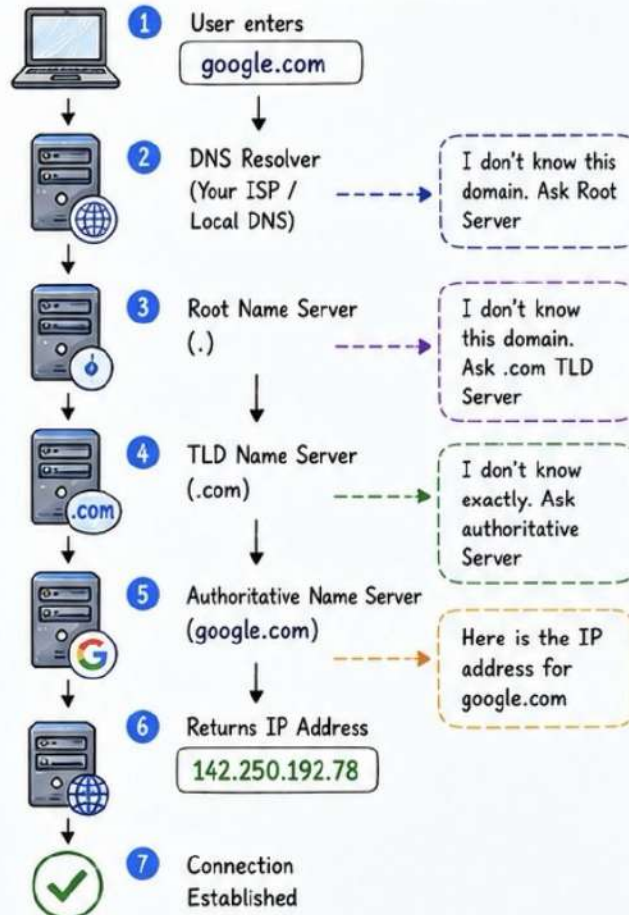
google.com

DNS translates it to

142.250.192.78

(IP Address)

DNS RESOLUTION FLOW



IMPORTANT DNS RECORDS



A RECORD

Maps domain to IPv4 address

Example: google.com → 142.250.192.78



AAAA RECORD

Maps domain to IPv6 address

Example: google.com → 2404:6800:4005:80b::200e



CNAME RECORD

Alias / points one domain to another domain

Example: www → google.com



MX RECORD

Mail exchange record (email routing)

Example: google.com → smtp.google.com



TXT RECORD

Text information (verification, SPF, DKIM, etc.)

Example: v=spf1 include:_spf.google.com ~all



NS RECORD

Name servers for a domain

Example: ns1.google.com

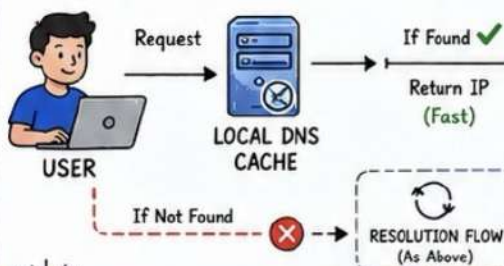


PTR RECORD

Reverse DNS (IP → Domain)

Example: 78.192.250.142 → google.com

DNS CACHING (SPEED BOOSTER)



QUICK TIP

- ✓ DNS issues = website not opening? Check using: nslookup, dig, host
- ✓ Always set correct TTL for performance and reliability.

DNS TTL (TIME TO LIVE)



TTL decides how long a DNS record is cached.

Low TTL → Frequent lookups
High TTL → Fewer lookups (Faster)

Common TTL Values:
60s, 300s, 3600s, 86400s (1 day)

COMMON TOOLS



nslookup



dig



host



DNS Checker

TYPES OF DNS



Public DNS

Google DNS (8.8.8.8, 8.8.4.4)
Cloudflare DNS (1.1.1.1)



Private DNS

Used inside organizations / VPCs



Authoritative DNS

Holds original DNS records



Recursive DNS

Resolves queries on behalf of clients

INTERVIEW QUESTION



Can you explain the complete DNS resolution process step by step?

What happens if DNS is down?



04
/10

HTTP vs HTTPS

4/10

DEVOPS
SHACK

Secure Communication Over The Internet

HTTP (INSECURE)

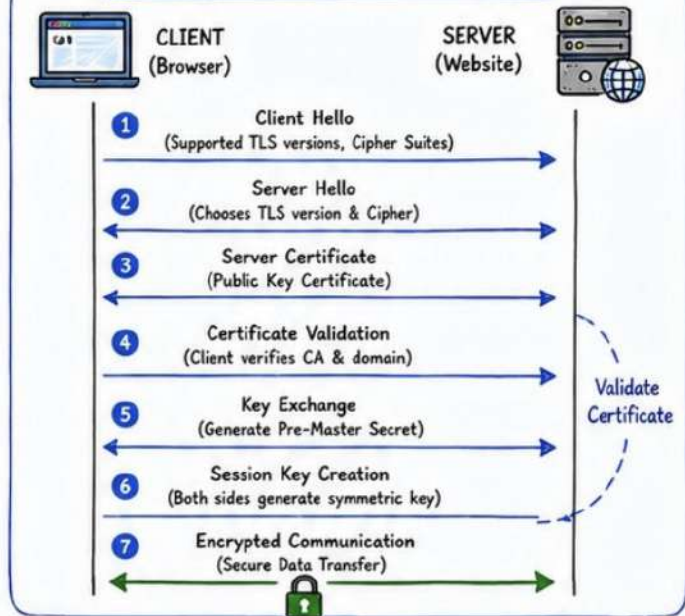
- Port: 80
- No Encryption
- Data in Plain Text
- Vulnerable to Attacks
- Not Safe for Sensitive Data
- Faster but Risky

VS

HTTPS (SECURE)

- Port: 443
- Encrypted (SSL/TLS)
- Data is Encrypted
- Protects from Attacks
- Safe for Sensitive Information
- Slightly Slower But Much Safer

TLS HANDSHAKE FLOW (HTTPS)



HTTPS = HTTP + SSL/TLS (Security Layer)

INSECURE HTTP CONNECTION



SECURE HTTPS CONNECTION



PROBLEMS WITH HTTP

- DATA SNIFFING**
Attackers can capture network packets and read sensitive data like passwords, cookies, etc.
- MAN-IN-THE-MIDDLE (MITM)**
Attackers can intercept and modify communication between client and server.
- DATA TAMPERING**
Attackers can modify data in transit without detection.
- NO AUTHENTICATION**
No way to verify if the server is genuine or fake.

BENEFITS OF HTTPS

- ENCRYPTION**
Data is encrypted using SSL/TLS, preventing unauthorized access.
- AUTHENTICATION**
Verifies the identity of the website using trusted SSL certificates.
- DATA INTEGRITY**
Ensures data is not altered during transmission.
- SEO BOOST**
Google ranks HTTPS sites higher than HTTP sites.
- USER TRUST**
Shows padlock in browser, increasing user trust.

HOW TO CHECK?

- Look for the padlock icon in the address bar.
- URL should start with https://
- Click on the padlock to view certificate details.

COMMON PORTS

HTTP	80	FTP	21
HTTPS	443	SSH	22
		DNS	53

QUICK TIP

Always use HTTPS for websites, APIs, logins, payments and any communication involving sensitive data.

DEVOPS SHACK

PRODUCTION-GRADE DEVOPS • CLOUD • KUBERNETES • DEVSECOPS

FOLLOW US FOR MORE



05
/10

LOAD BALANCERS & REVERSE PROXIES

5/10
DEVOPS
SHACK

Traffic Distribution & Application Delivery

WHAT IS A LOAD BALANCER?

A Load Balancer distributes incoming traffic across multiple servers to ensure no single server is overwhelmed, improving availability, scalability and reliability.

High
Availability

Scalability

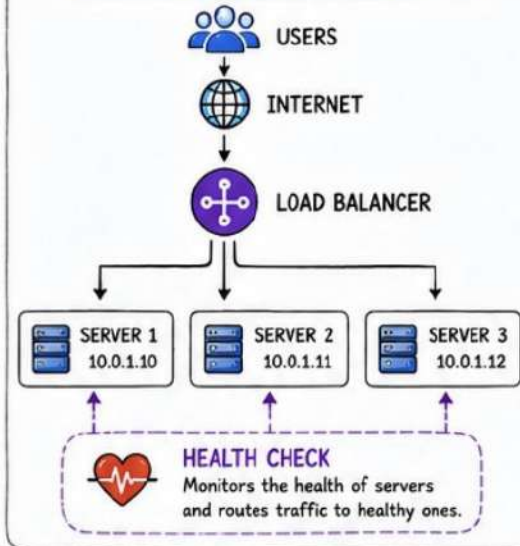


Reliability



Performance

LOAD BALANCER ARCHITECTURE



TYPES OF LOAD BALANCERS



AWS ALB (APPLICATION LOAD BALANCER)

- Operates at Layer 7 (Application Layer)
- HTTP/HTTPS traffic
- Advanced routing (host/path based)
- Ideal for web applications



AWS NLB (NETWORK LOAD BALANCER)

- Operates at Layer 4 (Transport Layer)
- TCP, UDP, TLS traffic
- Extreme performance
- Ideal for high throughput



CLASSIC LOAD BALANCER

- Legacy Load Balancer
- Supports HTTP, HTTPS, TCP, SSL
- Basic routing

COMMON LOAD BALANCING ALGORITHMS



ROUND ROBIN

Distributes traffic evenly across all servers.



LEAST CONNECTIONS

Sends traffic to the server with the least active connections.



IP HASH

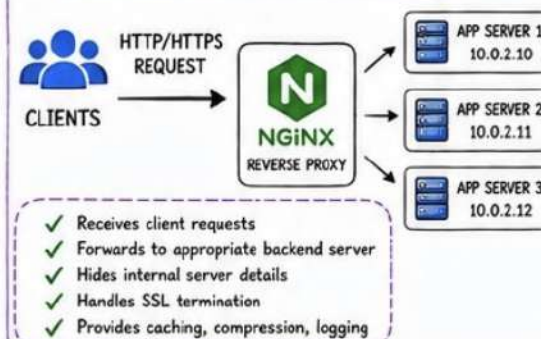
Routes requests from the same IP to the same server.



WEIGHTED ROUND ROBIN

Distributes traffic based on server weights.

REVERSE PROXY ARCHITECTURE



POPULAR REVERSE PROXIES



NGINX

- High performance
- Widely used
- Rich feature set



TAEFIK

- Modern & dynamic
- Great for Docker & K8s
- Automatic service discovery



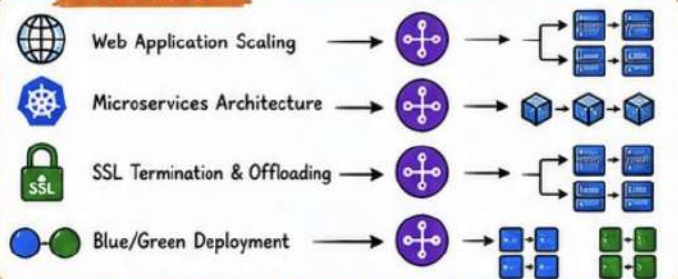
HA PROXY

- High availability
- Layer 4 & 7 proxy
- Very reliable

LOAD BALANCER VS REVERSE PROXY

FEATURE	LOAD BALANCER	REVERSE PROXY
Primary Role	Distributes traffic across servers	Sits in front of servers and forwards requests
OSI Layer	Layer 4 or Layer 7	Layer 7 (Application Layer)
Client Awareness	Clients connect to LB directly	Clients connect to Reverse Proxy
SSL Termination	Yes (Some LB's)	Yes (Commonly used)
Use Case	Scaling, High availability	Security, Routing, Caching, SSL
Examples	AWS ALB, NLB, HAProxy (LB mode)	NGINX, Traefik, HAProxy (Proxy mode)

USE CASES



QUICK TIP

- ✓ Use ALB for web apps (HTTP/HTTPS)
- ✓ Use NLB for high performance (TCP/UDP)
- ✓ Use Reverse Proxy for security, caching and routing
- ✓ Always enable Health Checks!

TOOLS YOU WILL LOVE



NGINX



HAProxy



traefik



AWS ALB



AWS NLB

INTERVIEW QUESTION



What is the difference between Load Balancer and Reverse Proxy? When would you use one over the other?



DEVOPS SHACK

PRODUCTION-GRADE DEVOPS
CLOUD • KUBERNETES • DEVSECOPS

FOLLOW US FOR MORE



08
/10

KUBERNETES NETWORKING

8/10

DEVOPS
SHACK

How Pods, Services & Ingress Communicate

K8S NETWORKING FUNDAMENTALS



Every Pod Gets an IP
Each pod gets a unique IP address in the cluster.



Flat Network Model
All pods can communicate with each other across nodes without NAT.



Pod-to-Pod Communication
Pods can talk to each other directly using IP.



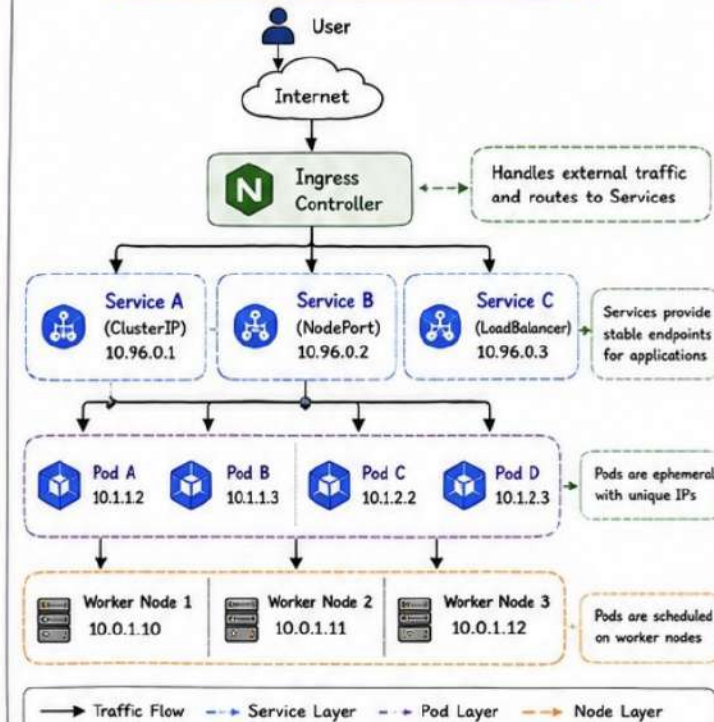
Service Abstraction
Services provide a stable endpoint (IP + DNS) for pods.

POD NETWORKING



- ✓ All pods are on the same flat network
- ✓ No NAT between pods

KUBERNETES NETWORKING ARCHITECTURE



KUBERNETES DNS

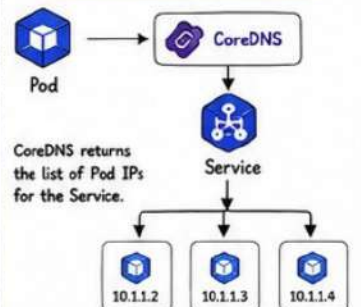
**CoreDNS**

Cluster DNS: 10.96.0.10

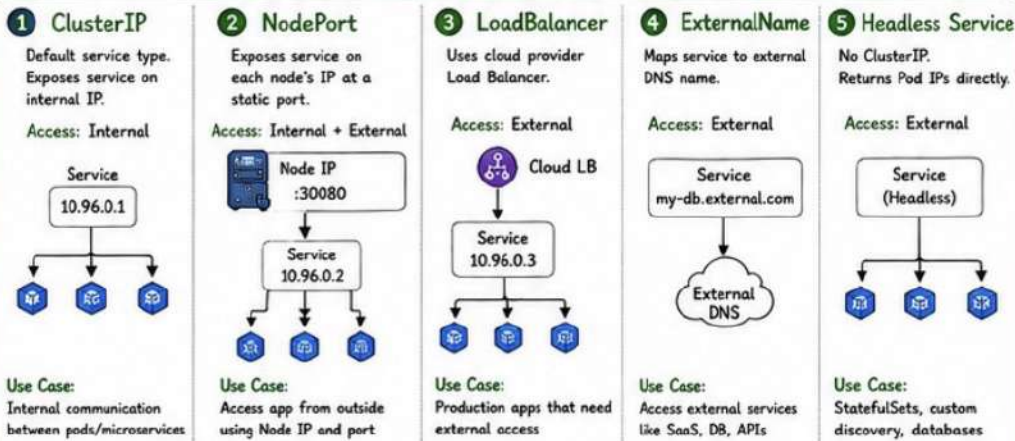
- ✓ Resolves Service names to ClusterIP
- ✓ Resolves Pod names within namespace
- ✓ Example:

`http://service-a.default.svc.cluster.local`

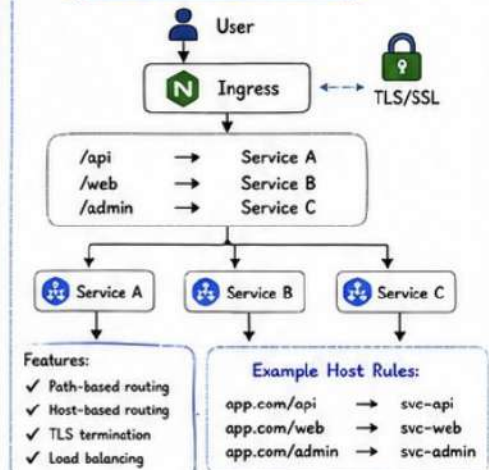
SERVICE DISCOVERY



KUBERNETES SERVICE TYPES



INGRESS ARCHITECTURE



CNI PLUGINS (NETWORK PROVIDERS)



These plugins handle pod networking, routing, and network policies in Kubernetes.

NETWORK POLICIES



- ✓ Control traffic flow at pod level.
- ✓ Allow or deny traffic based on rules.
- ✓ Implemented using CNI plugins.

Example: Allow only frontend pods to access backend pods on port 8080.

COMMON PORTS IN K8S

53	DNS (CoreDNS)
80	HTTP
443	HTTPS
10250	Kubelet (Node)
2379	etcd
6443	Kubernetes API
30000-32767	NodePort Range

INTERVIEW QUESTION



How does a pod in one node communicate with a pod in another node?

Think about CNI, routing, and flat network model!



09
/10

NETWORK TROUBLESHOOTING & DEBUGGING

9/10

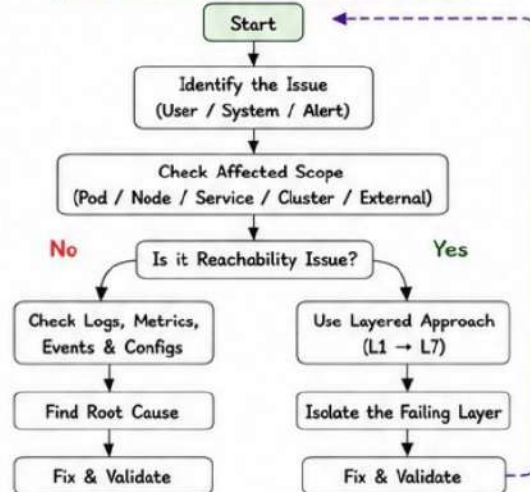
SHACK

A Systematic Approach to Find & Fix Network Issues

TROUBLESHOOTING MINDSET

- 1. DEFINE THE PROBLEM**
What is not working?
What is the expected behavior?
- 2. GATHER INFORMATION**
Collect logs, metrics, configs and topology.
- 3. FORM HYPOTHESES**
What could be causing it?
- 4. TEST & ISOLATE**
Test each layer to narrow down the issue.
- 5. RESOLVE & VALIDATE**
Fix the root cause and verify the solution.

NETWORK TROUBLESHOOTING FLOW



THE 7 LAYERS CHECKLIST

- L7 Application**
(HTTP, DNS, gRPC, etc.)
- L6 Presentation**
(SSL/TLS, Encryption)
- L5 Session**
(NetBIOS, RPC, etc.)
- L4 Transport**
(TCP, UDP)
- L3 Network**
(IP, Routing)
- L2 Data Link**
(MAC, VLAN)
- L1 Physical**
(Cable, NIC, Signal)

ESSENTIAL NETWORK DEBUGGING TOOLS

ping



Check basic connectivity (L3)

traceroute



Trace the path packets take (L3)

nslookup
dig

DNS lookup and resolution (L7)

curl
wget

Test application connectivity (L7)

netstat
ss

Check open ports and connections (L4)

tcpdump



Capture and analyze packets (L2-L4)

iptables
nftables

Check firewall rules (L3-L4)

kubectl
logs/describe

Check K8s resources and events

PRO TIP

- ✓ Start from the user side and move inward.
- ✓ Check one layer at a time.
- ✓ Compare working vs not working.
- ✓ Logs don't lie!
- ✓ Reproduce the issue consistently.

COMMON NETWORK ISSUES & SYMPTOMS

- Cannot reach service
→ DNS / Routing / Security Group
- Intermittent timeout
→ Network congestion / MTU / Firewall
- Slow response
→ High latency / Packet loss / CPU
- Connection refused
→ Service down / Wrong port / Firewall
- SSL/TLS errors
→ Cert issues / TLS mismatch / Clock skew
- DNS resolution failure
→ CoreDNS / DNS records / Network
- Pod to Pod not working
→ CNI / Network Policy / Calico

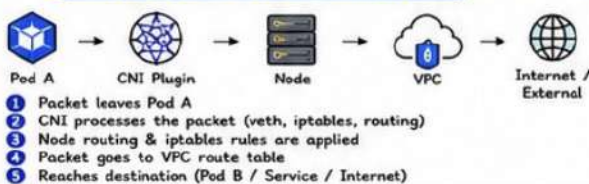
POD CONNECTIVITY CHECKS

-
- Are both pods running?
`kubectl get pods`
 - Can Pod A ping Pod B IP?
`kubectl exec -it pod-a -- ping <pod-b-ip>`
 - Can Pod A reach Pod B port?
`kubectl exec -it pod-a -- nc -zv <pod-b-ip> <port>`
 - Check Network Policy
`kubectl get networkpolicies -A`
 - Check CNI & Logs
`kubectl -n kube-system logs -l kube-apiserver=cni`

NETWORK POLICY DEBUGGING

-
- ✓ List all policies in the namespace
`kubectl get networkpolicies -n <ns> -o wide`
 - ✓ Describe the policy
`kubectl describe networkpolicy <policy> -n <ns>`
 - ✓ Verify selectors
Do pod labels match the policy selectors?
 - ✓ Check ingress/egress rules
Are the allowed sources/destinations correct?
 - ✓ Remember: Default Deny = Block All

PACKET FLOW PATH (EXAMPLE)



CHECKLIST BEFORE ESCALATION

- ✓ Reproduced the issue consistently
- ✓ Checked all 7 OSI layers
- ✓ Verified configuration & policies
- ✓ Collected logs, events & metrics
- ✓ Identified the failing point
- ✓ Documented findings clearly

INTERVIEW QUESTION

- A pod in one namespace cannot access a service in another namespace.
- How would you troubleshoot it?

QUICK REFERENCE COMMANDS

Check Pod IP
`kubectl get pod -o wide`

Describe Pod
`kubectl describe pod <pod>`

Check Services
`kubectl get svc -A`

Check Endpoints
`kubectl get ep -A`

Check Nodes
`kubectl get nodes -o wide`

Check CNI Pods
`kubectl -n kube-system`



QUICK TIP
Document your troubleshooting steps. It helps next time!



DEVOPS SHACK
PRODUCTION-GRADE DEVOPS
CLOUD • KUBERNETES • DEVSECOPS

FOLLOW US FOR MORE



10
/10

&

CLOUD NETWORKING & SECURITY BEST PRACTICES

10/10

SHACK

Design Secure, Scalable & Resilient Cloud Networks

1. DESIGN PRINCIPLES



Security by Design

Embed security in every layer from the start.



Least Privilege

Grant minimum permissions required to perform a task.



Defense in Depth

Use multiple layers of security controls.



High Availability

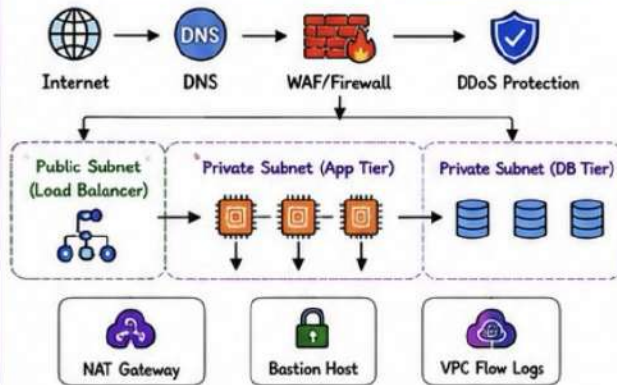
Eliminate single points of failure. Use redundancy.



Scalability

Design for growth. Auto-scale when needed.

2. CLOUD NETWORK ARCHITECTURE BEST PRACTICES



- ✓ Use VPC/VNet to isolate your resources.
- ✓ Place workloads in private subnets.
- ✓ Use NAT Gateways for outbound internet access.
- ✓ Use Bastion Host for secure admin access.
- ✓ Enable VPC Flow Logs for visibility.

3. NETWORK SECURITY CONTROLS



Security Groups / NSGs

Control inbound and outbound traffic at instance/subnet level.



Network ACLs

Stateless filtering at subnet level (allow/deny rules).



Firewalls

Inspect and filter traffic based on rules.



Web Application Firewall (WAF)

Protect web applications from OWASP top 10 threats.



DDoS Protection

Enable DDoS protection to absorb and mitigate attacks.



IDS/IPS

Detect and prevent malicious activities in the network.

4. SECURE CONNECTIVITY



VPN / Direct Connect

Use encrypted tunnels for secure connectivity to on-prem.



Private Link / Private Endpoint

Access cloud services over private network.



Peering

Use VPC peering for private communication between networks.



TLS Everywhere

Encrypt data in transit using TLS/SSL.

5. MONITORING & LOGGING



Enable Flow Logs

Capture network traffic metadata.



Centralize Logs

Send logs to centralized logging solution.



Set Up Alerts

Create alerts for suspicious activities.



Dashboards

Monitor traffic, errors, and threats in real-time.



Regular Audits

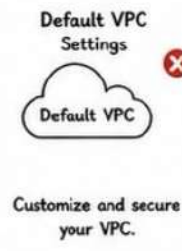
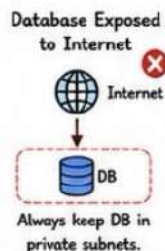
Review logs and network configurations periodically.

6. CONFIGURATION BEST PRACTICES

- ✓ Keep Security Groups / NSGs rules minimal.
- ✓ Deny all by default. Allow only necessary traffic.
- ✓ Use meaningful tags for resources.
- ✓ Enable multi-factor authentication (MFA).
- ✓ Regularly update and patch systems.
- ✓ Use infrastructure as code (IaC) for consistency.
- ✓ Backup configurations and test recovery.
- ✓ Disable unused ports and services.

7. COMMON PITFALLS TO AVOID

- ✗ Exposing databases to the internet.
- ✗ Overly permissive security group rules (0.0.0.0/0).
- ✗ Not enabling network flow logs.
- ✗ Using default VPC settings.
- ✗ Ignoring egress traffic filtering.
- ✗ No monitoring or alerting setup.
- ✗ Hardcoding IPs and credentials.
- ✗ Not testing security controls.



BEST PRACTICE SUMMARY



- ✓ Designing with security as a priority.
- ✓ Follow least privilege and deny by default.
- ✓ Use private subnets for workloads.
- ✓ Encrypt data in transit and at rest.
- ✓ Monitor, log, and audit continuously.
- ✓ Automate and validate configurations.



QUICK TIP

Always assume breach. Validate, monitor, and continuously improve your network security.



INTERVIEW QUESTION



What are the key components of a secure cloud network architecture?

What is the difference between Security Groups and Network ACLs?



USEFUL COMMANDS



- # List security groups (AWS)
`aws ec2 describe-security-groups`
- # VPC Flow Logs (AWS CLI)
`aws ec2 describe-flow-logs`
- # List network interfaces
`aws ec2 describe-network-interfaces`